

STUDENT ROLL NUMBER NAME Sajawal Shahbaz (23010101-049)
Ehtesham hafeez (23010101-094)

GRADE POINT ALLOCATED

--

Instructor: Museb Khalid

Medical Report Analyzer

Project Report

What this project is?

Medical Report Analyzer is a dashboard-style app that helps someone keep track of their medical reports blood tests, X-rays, MRIs, ECGs in one place instead of digging through paper files or random PDFs every time. A user uploads a report along with basic patient details, and the app is meant to show a clean summary of the important numbers (blood pressure, sugar level, heart rate, oxygen level) and keep a history of everything that was uploaded before.

This deliverable is only about the frontend. The actual analysis logic, authentication, and storage will be connected later once the backend (Express + MongoDB) is added in the next phase, so right now the app just runs on sample/local data to prove the interface and navigation work properly.

Frontend (Built with Vue 3 + Vite):

The frontend is a single-page application built with Vue 3 and Vite. Instead of separate HTML pages for every screen, the whole thing is one app that swaps out components on the fly, which is what makes it feel instant when moving between pages.

Vue Router Or Page Navigation:

Vue Router handles moving between all the main pages Home, Dashboard, Login, Register, Upload, Analysis, History and Profile without ever reloading the browser. Every route is registered in one place, `src/router/index.js`, and I used history mode so the URLs look normal (like `/dashboard`) instead of having a `#` in them.

The layout itself lives in `App.vue`, where the Navbar, Sidebar and Footer stay fixed on every page, and only the middle part changes using `<router-view>`, wrapped in a transition so pages fade in instead of just popping in.

Component Structure:

The project is split into two folders. `components/` holds pieces that get reused Navbar, Sidebar, Footer, DashboardChart, FeatureCard, CustomButton and `views/` holds one file per page/route (Home, Dashboard, Login, Register, Upload, Analysis, History, Profile, NotFound). This keeps every view focused on just its own page, and the shared stuff like the navbar never has to be repeated.

Props and Emits:

To actually show data flowing between a parent and a child component, I built two small reusable components. `FeatureCard.vue` just displays whatever it's given an icon, a title and a description are passed in as props from the parent, so the card itself doesn't need to know anything, it just renders what it receives.

`CustomButton.vue` works the other way around. It doesn't decide what happens when it's clicked it just emits a button-click event and lets the parent decide what to do with it. That way the same button can be reused anywhere and just wired up differently each time.

Put together, a parent view would use them like this passing data down through props, and listening for the click going back up:

Other Reusable Components:

`DashboardChart.vue` wraps a `Chart.js` bar chart and is dropped straight into the Dashboard page to show monthly report counts. `Navbar.vue`, `Sidebar.vue` and `Footer.vue` are the shell pieces that stay on screen no matter which page is active, so they only ever get written once.

Local State:

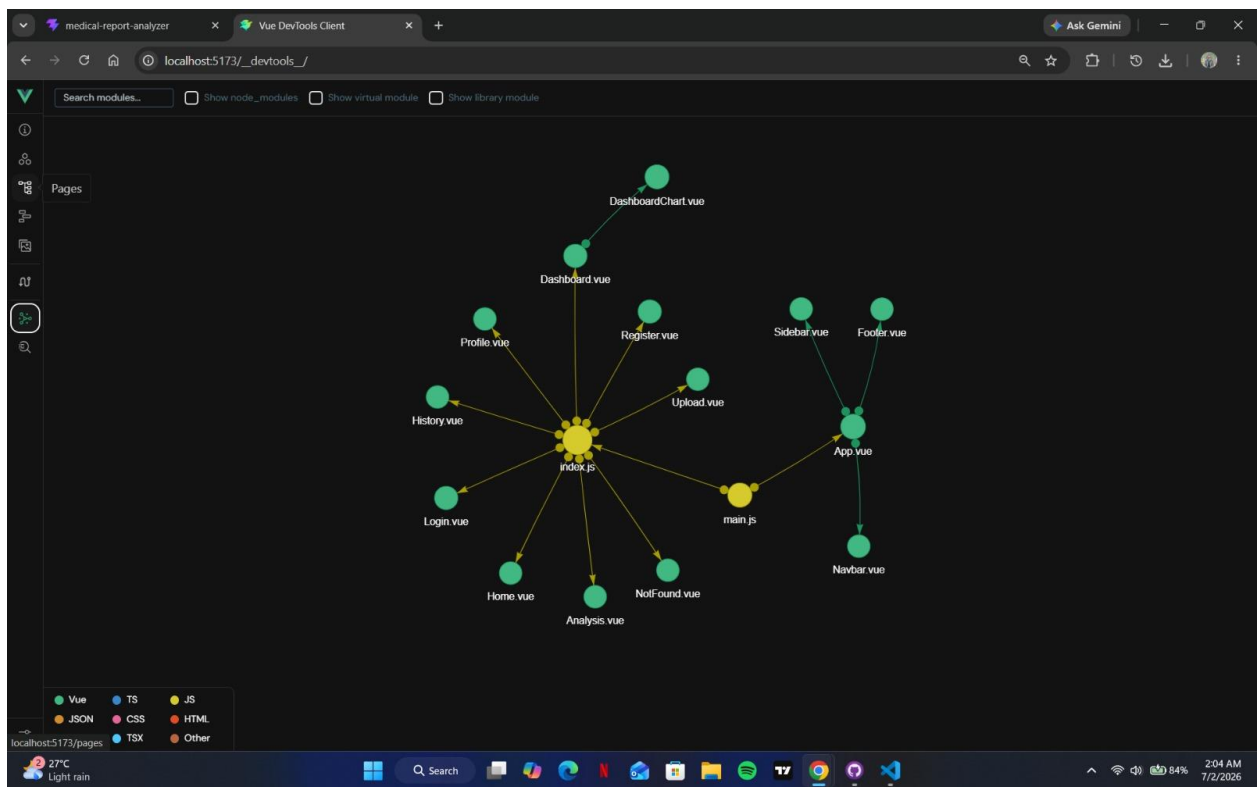
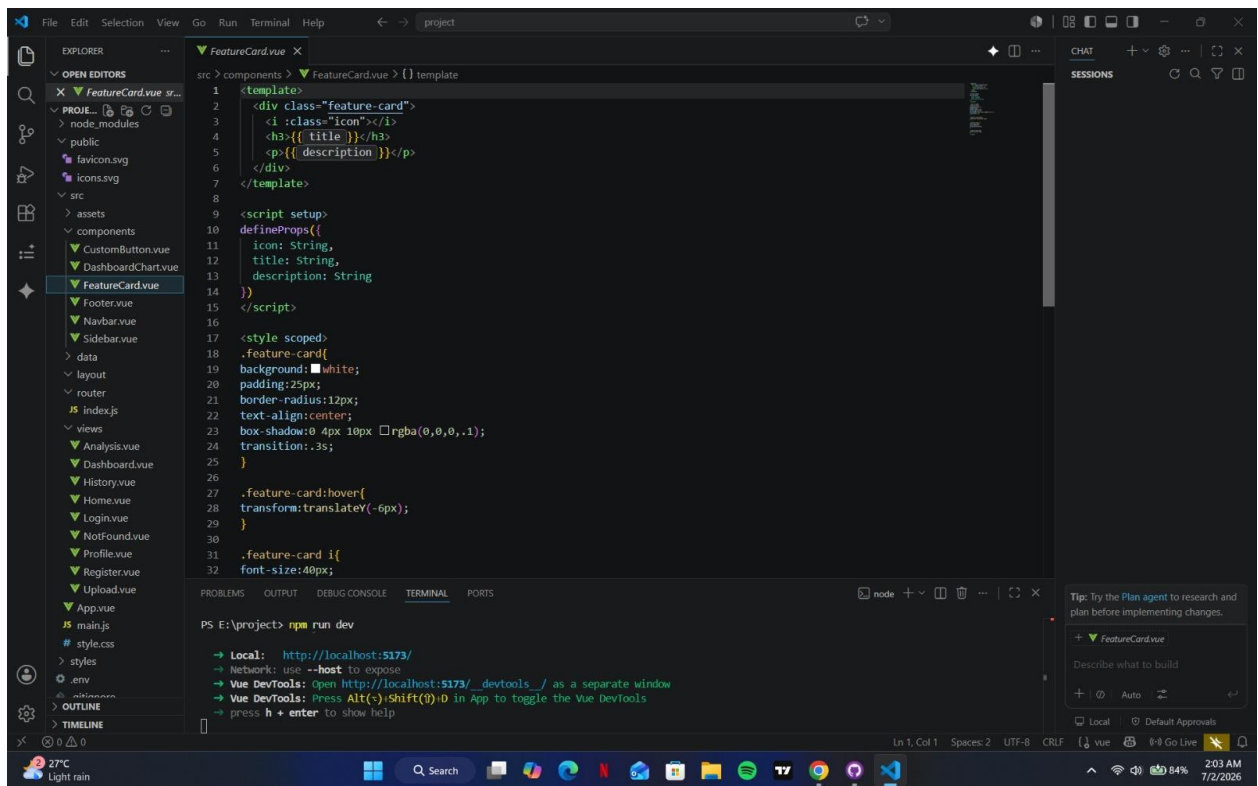
Anything that doesn't need to be shared across components is just kept as local state using Vue's `ref()`. For example, `Navbar.vue` keeps its own dark-mode toggle in a `ref`, and `Upload.vue` keeps every form field (patient name, age, gender, report type, hospital, doctor, notes) as its own `ref` bound with `v-model`, only collecting everything together when the form is submitted.

Styling:

Styling is done with a shared global CSS file imported once in `main.js`, so every page and component ends up looking consistent without repeating the same colors and spacing everywhere.

Component Architecture:

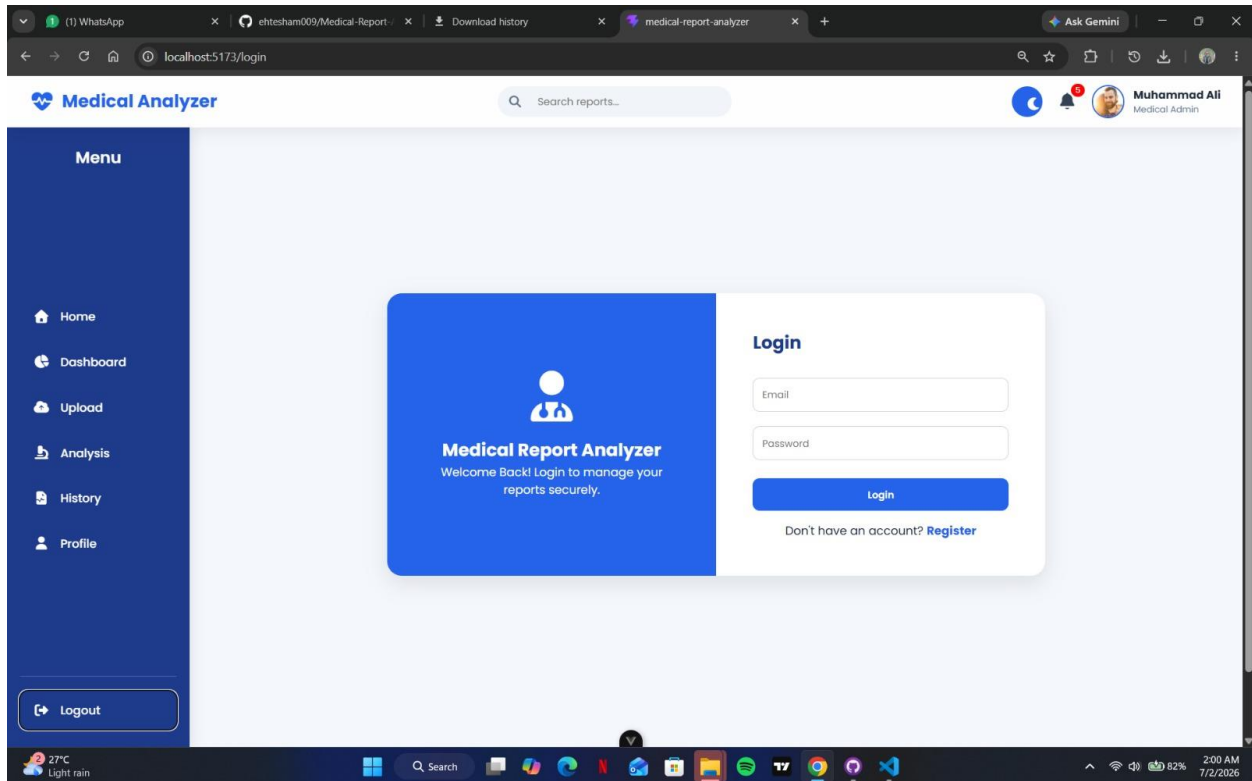
Here's a simple diagram of how the components fit together the shell (`Navbar`, `Sidebar`, `Footer`) sits around whatever page the router is currently showing, and the reusable widgets (`FeatureCard`, `CustomButton`, `DashboardChart`) can be dropped into any view that needs them.



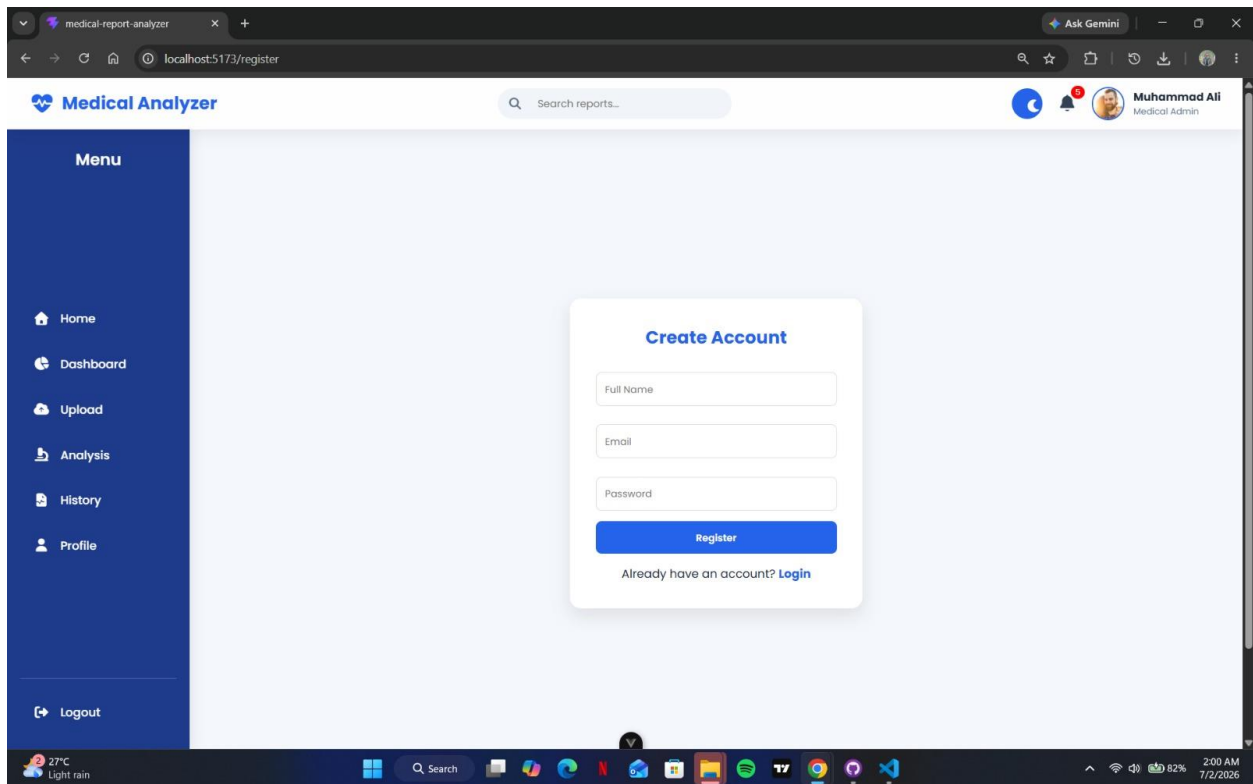
UI Screenshots:

The app was run locally with `npm run dev`, which serves it at `http://localhost:5173/`. These screenshots show the interface actually running.

Login page:



Register Page:



The screenshot shows the 'Register' page of the 'Medical Analyzer' application. The browser address bar shows 'localhost:5173/register'. The page features a dark blue sidebar menu on the left with options: Home, Dashboard, Upload, Analysis, History, Profile, and Logout. The main content area has a light blue background with a 'Create Account' form in the center. The form includes input fields for 'Full Name', 'Email', and 'Password', followed by a blue 'Register' button. Below the button is a link: 'Already have an account? Login'. The top right corner shows the user 'Muhammad Ali' as 'Medical Admin'. The bottom of the screen shows a Windows taskbar with various icons and a system tray indicating 27°C and light rain.

Medical Analyzer

Search reports...

Menu

- Home
- Dashboard
- Upload
- Analysis
- History
- Profile
- Logout

Create Account

Full Name

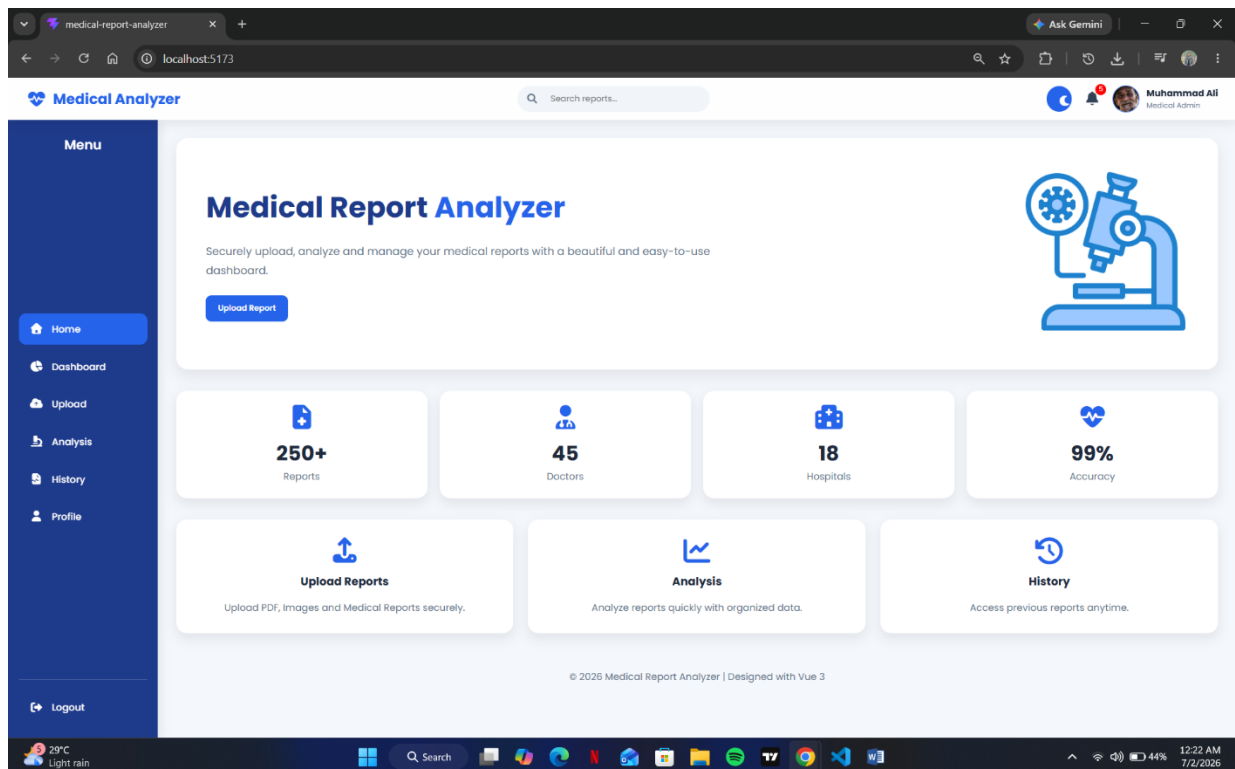
Email

Password

Register

Already have an account? [Login](#)

Home Page:



The screenshot shows the 'Home' page of the 'Medical Analyzer' application. The browser address bar shows 'localhost:5173'. The page features a dark blue sidebar menu on the left with options: Home, Dashboard, Upload, Analysis, History, Profile, and Logout. The main content area has a light blue background with a 'Medical Report Analyzer' header and a description: 'Securely upload, analyze and manage your medical reports with a beautiful and easy-to-use dashboard.' Below the header is a blue 'Upload Report' button. The page displays four statistics: 250+ Reports, 45 Doctors, 18 Hospitals, and 99% Accuracy. Below these are three action cards: 'Upload Reports' (Upload PDF, Images and Medical Reports securely.), 'Analysis' (Analyze reports quickly with organized data.), and 'History' (Access previous reports anytime.). The bottom of the screen shows a Windows taskbar with various icons and a system tray indicating 29°C and light rain.

Medical Analyzer

Search reports...

Menu

- Home
- Dashboard
- Upload
- Analysis
- History
- Profile
- Logout

Medical Report Analyzer

Securely upload, analyze and manage your medical reports with a beautiful and easy-to-use dashboard.

Upload Report

250+ Reports

45 Doctors

18 Hospitals

99% Accuracy

Upload Reports

Upload PDF, Images and Medical Reports securely.

Analysis

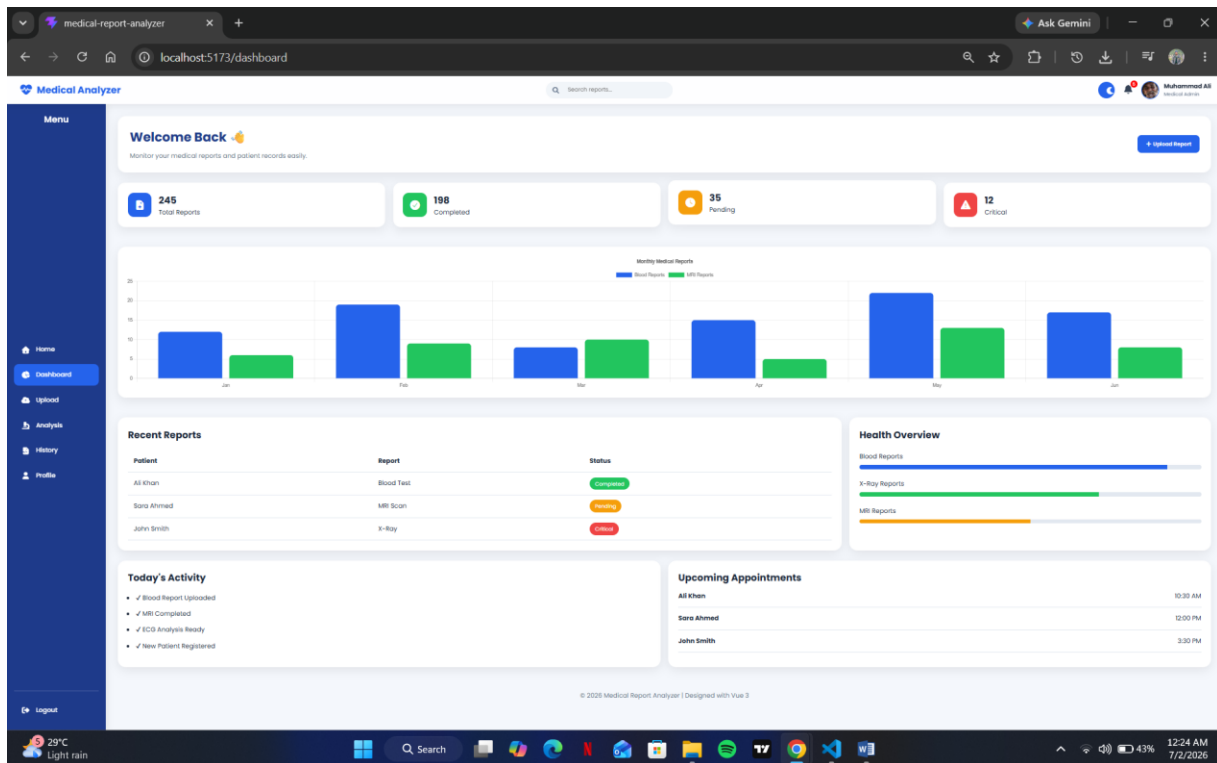
Analyze reports quickly with organized data.

History

Access previous reports anytime.

© 2026 Medical Report Analyzer | Designed with Vue 3

Dashboard Page:



Upload Page:

The upload page features a sidebar menu, a search bar, and a main content area with a drag-and-drop report upload section and a form for patient details and report information.

Upload Medical Report

Fill in the patient details and upload the medical report securely.

Drag & Drop Report

PDF, JPG, PNG (Max 10MB)

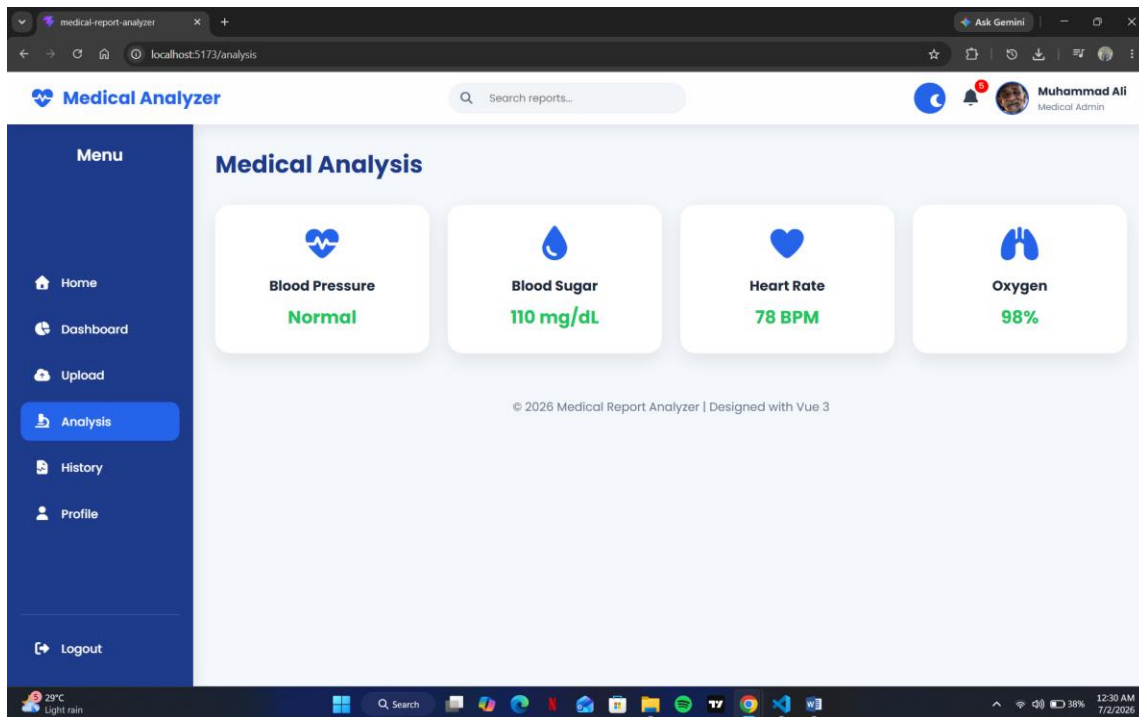
Form Fields:

- Patient Name:
- Age:
- Gender:
- Report Type:
- Hospital Name:
- Doctor Name:
- Notes:

Buttons:

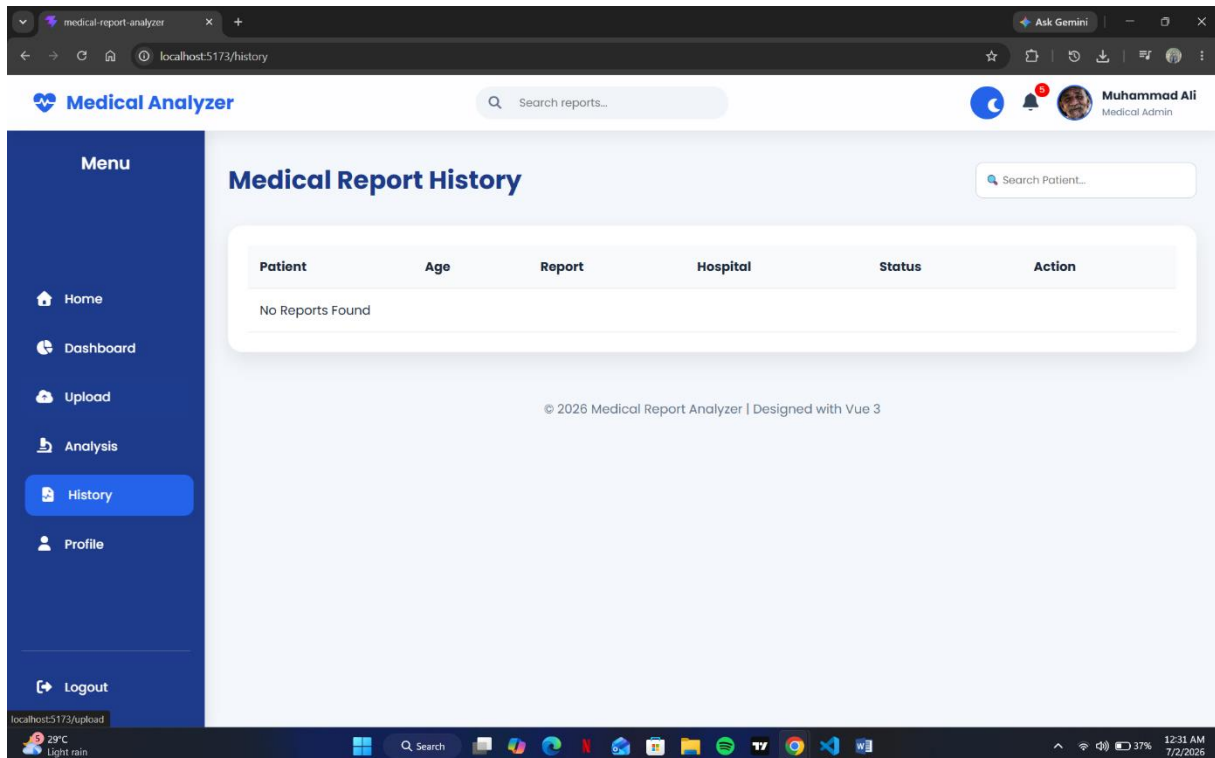
-
-

Analysis Page:



The screenshot shows the 'Analysis' page of the Medical Analyzer application. The browser address bar displays 'localhost:5173/analysis'. The page features a dark blue sidebar menu with options: Home, Dashboard, Upload, Analysis (highlighted), History, Profile, and Logout. The main content area is titled 'Medical Analysis' and contains four white cards with blue icons and green text: 'Blood Pressure Normal', 'Blood Sugar 110 mg/dL', 'Heart Rate 78 BPM', and 'Oxygen 98%'. A search bar labeled 'Search reports...' is at the top right. The user profile 'Muhammad Ali Medical Admin' is in the top right corner. The footer shows '© 2026 Medical Report Analyzer | Designed with Vue 3'. The Windows taskbar at the bottom shows the date as 7/2/2026 and time as 12:30 AM.

History Page:



The screenshot shows the 'History' page of the Medical Analyzer application. The browser address bar displays 'localhost:5173/history'. The sidebar menu is identical to the Analysis page, with 'History' highlighted. The main content area is titled 'Medical Report History' and includes a 'Search Patient...' input field. Below this is a table with the following structure:

Patient	Age	Report	Hospital	Status	Action
No Reports Found					

The footer shows '© 2026 Medical Report Analyzer | Designed with Vue 3'. The Windows taskbar at the bottom shows the date as 7/2/2026 and time as 12:31 AM.

Profile Page:

